

M2.0

MENTORIA EM TESTES DE SOFTWARE

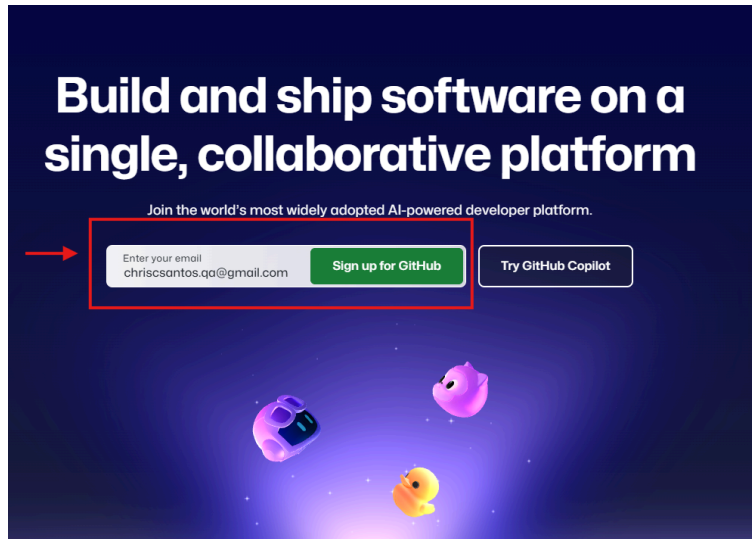
Tutorial Criando uma conta no GitHub e configurando um novo Repositório

1. Criando uma conta no GitHub

1.1 Acesse o site abaixo para iniciarmos a criação de uma conta no **GitHub**

<https://github.com/>

1.2 No campo “**Enter your email**” digite um e-mail válido e pessoal para utilizar como cadastro no **GitHub** e clique no botão “**Sign up for GitHub**”



1.3 Após isso **preencha os campos obrigatórios** do formulário com seus dados e clique no botão “**Continue**”

Sign up to GitHub

Email*

chriscsantos.qa@gmail.com ✓

Password*

..... ✓

Password should be at least 15 characters OR at least 8 characters including a number and a lowercase letter.

Username*

chriscsqa ✓

Username may only contain alphanumeric characters or single hyphens, and cannot begin or end with a hyphen.

Your Country/Region*

Brazil ▼

For compliance reasons, we're required to collect country information to send you occasional updates and announcements.

Email preferences

☐ Receive occasional product updates and announcements

Continue >

By creating an account, you agree to the [Terms of Service](#). For more information about GitHub's privacy practices, see the [GitHub Privacy Statement](#). We'll occasionally send you account-related emails.

1.4 Em seguida será necessário realizar um desafio de verificação de ação humana, clique no botão “**Puzzle visual**”, resolva o desafio conforme o mesmo pede e clique no botão “**Enviar**”

Verify your account

Resolva este desafio para sabermos que é uma pessoa

Puzzle visual

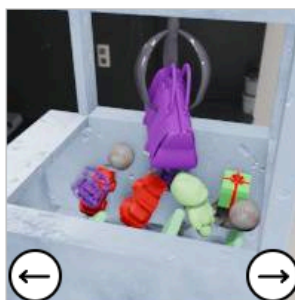
🔊 Áudio

Verify your account

Encontre a imagem que tem o prêmio à esquerda **segurado pela garra** na máquina à direita. (1 de 1)



Escolha o igual



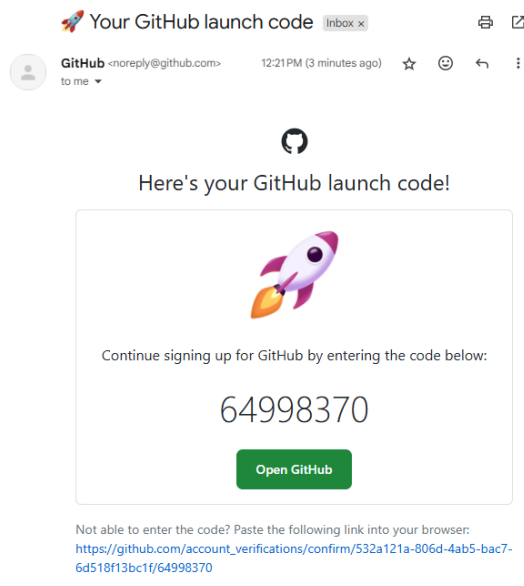
○○○○○○○●

Enviar

🔊 Áudio

🔄 Reiniciar

1.5 Após isso você irá receber um código no e-mail que utilizou para cadastro de conta



1.6 Copie o **código** recebido via e-mail e digite ele no cadastro do **GitHub**

Confirm your email address

We have sent a code to **chriscsantos.qa@gmail.com**

Enter code

6	4	9	9	8	3	7	0
---	---	---	---	---	---	---	---

[Continue](#)

Didn't get your email? [Resend the code](#) or [update your email address](#).

1.7 Após isso você será redirecionado a página de “**Sign in**” do **GitHub**, preencha os campos com seu **usuário** ou **email**, o **password** que você inseriu durante o cadastro e clique no botão “**Sign in**”.



Sign in to GitHub

Username or email address

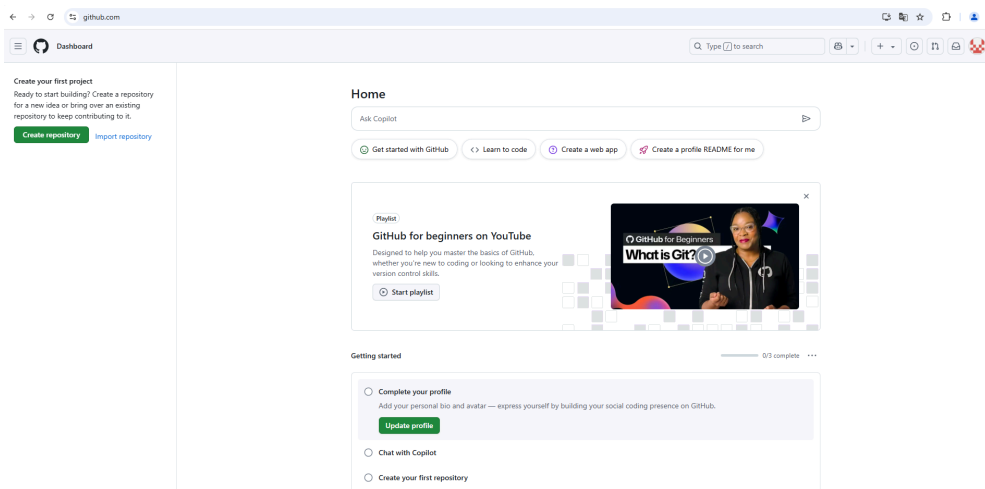
Password [Forgot password?](#)

Sign in

[Sign in with a passkey](#)

New to GitHub? [Create an account](#)

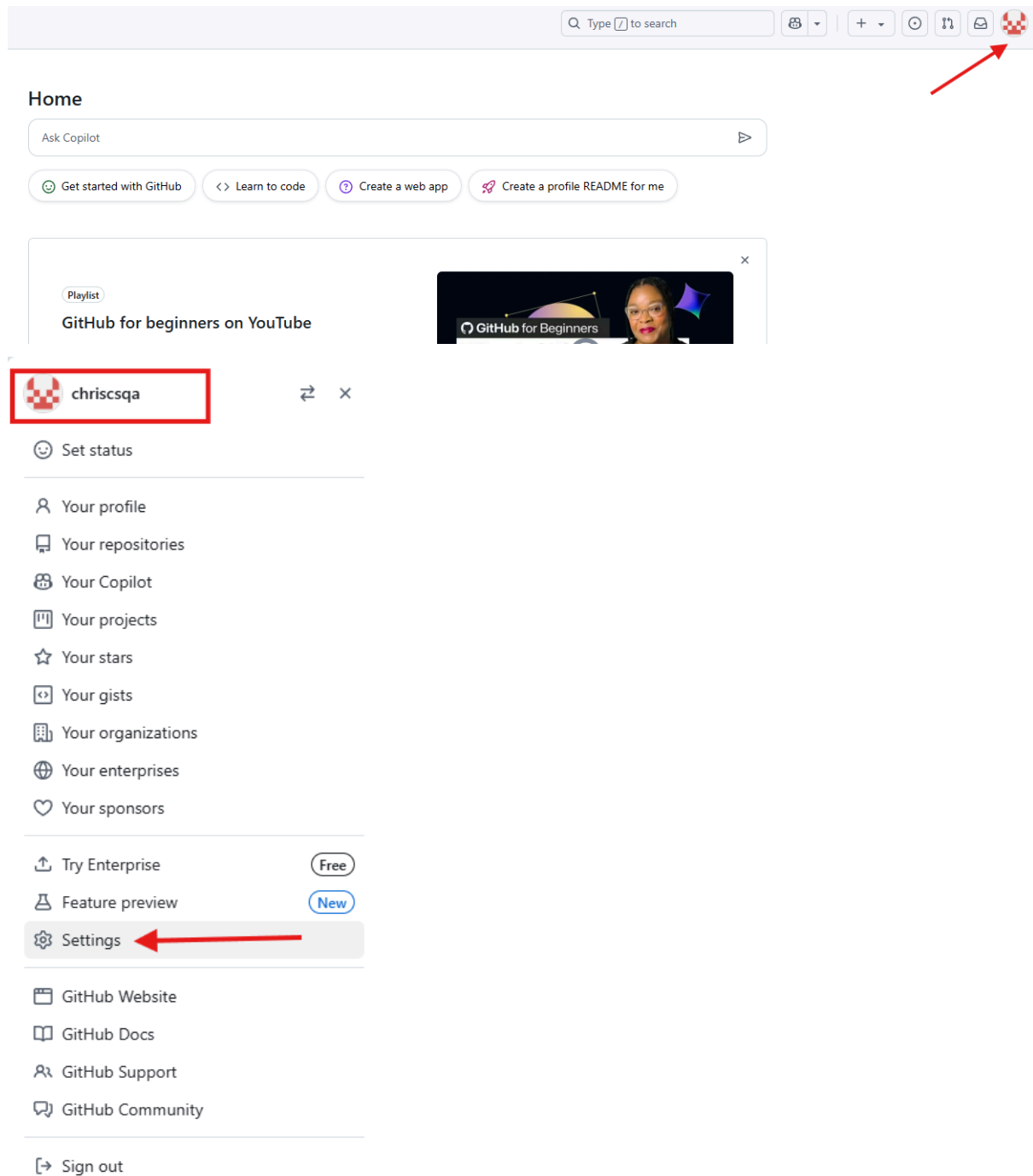
1.8 Pronto! Sua conta do **GitHub** está criada e agora você conseguiu acessar a plataforma com sucesso



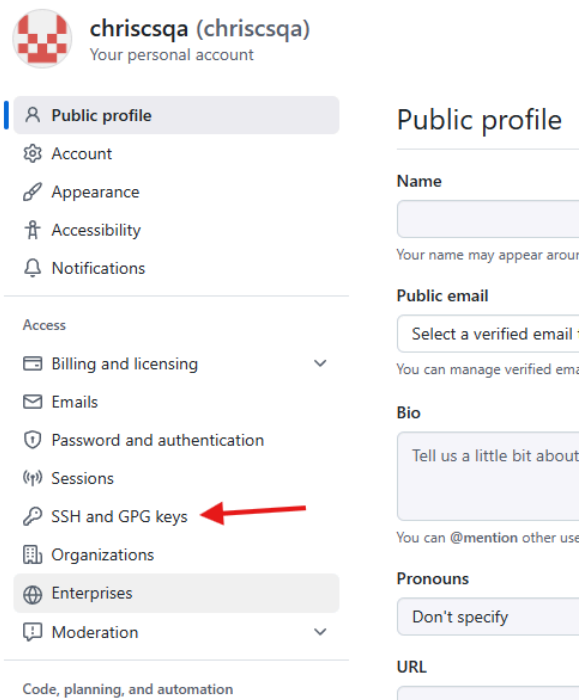
2. Configurando sua SSH com o GitHub

2.1 Agora vamos configurar uma chave **SSH** com o **GitHub** para que você consiga se comunicar com a plataforma e realizar o **clone**, **push**, **pull**, e **todas as ações** necessárias para você interagir com um repositório.

2.2 No canto superior da direita, clique em cima da sua foto de usuário e clique na opção **“Settings”** no menu que se abrir.

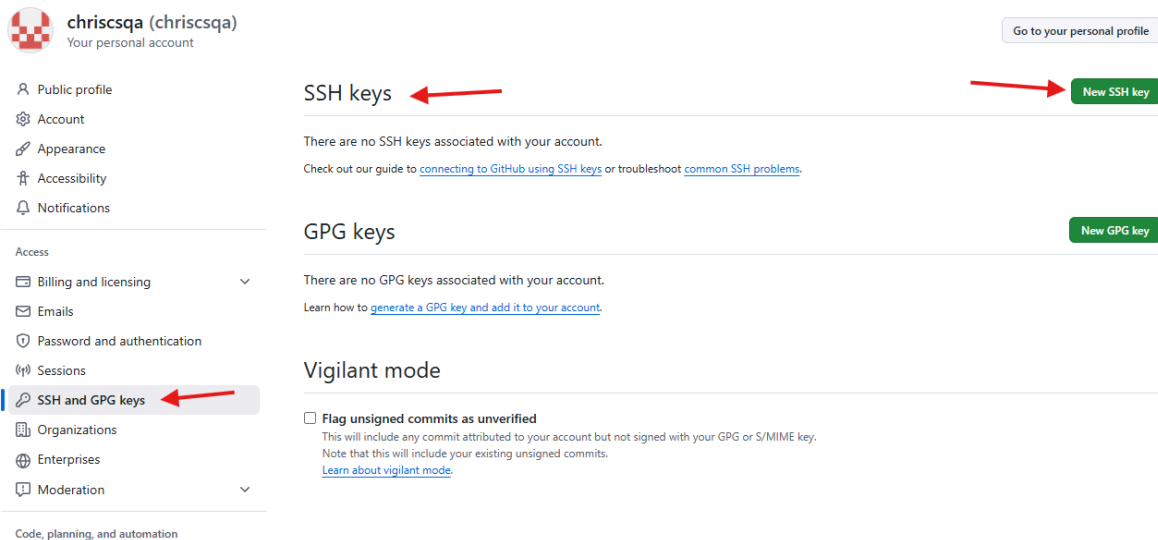


2.3 Agora na página que se abrir, clique em “**SSH and GPG keys**” no menu lateral da esquerda



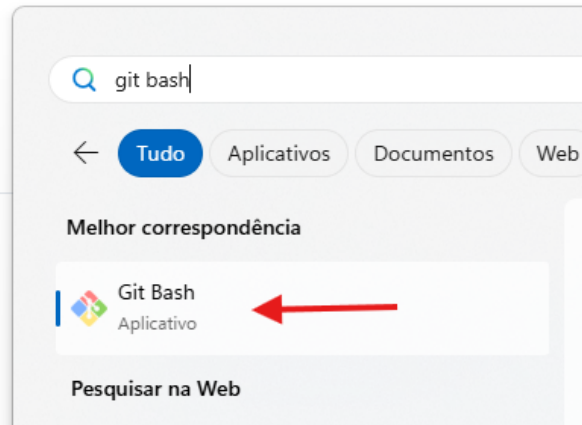
The screenshot shows the GitHub profile settings page for user 'chriscsqa'. On the left is a sidebar menu with options: Public profile, Account, Appearance, Accessibility, Notifications, Access, Billing and licensing, Emails, Password and authentication, Sessions, SSH and GPG keys (highlighted with a red arrow), Organizations, Enterprises, and Moderation. The main content area is titled 'Public profile' and contains fields for Name, Public email, Bio, Pronouns, and URL. The 'SSH and GPG keys' option in the sidebar is highlighted with a red arrow.

2.4 Agora na página que se abrir, clique no botão “**New SSH key**”

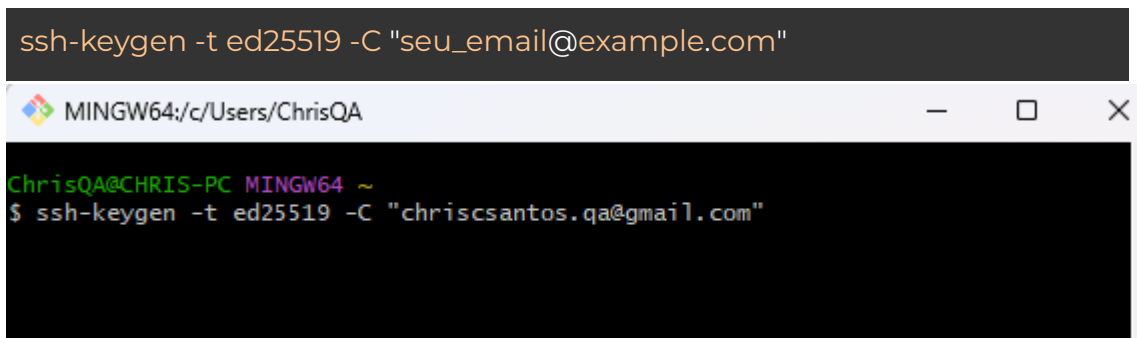


The screenshot shows the GitHub 'SSH and GPG keys' page for user 'chriscsqa'. The left sidebar menu is the same as in the previous screenshot, with 'SSH and GPG keys' highlighted by a red arrow. The main content area has three sections: 'SSH keys' (with a red arrow pointing to the 'SSH keys' title and a green 'New SSH key' button), 'GPG keys' (with a green 'New GPG key' button), and 'Vigilant mode' (with a checkbox for 'Flag unsigned commits as unverified').

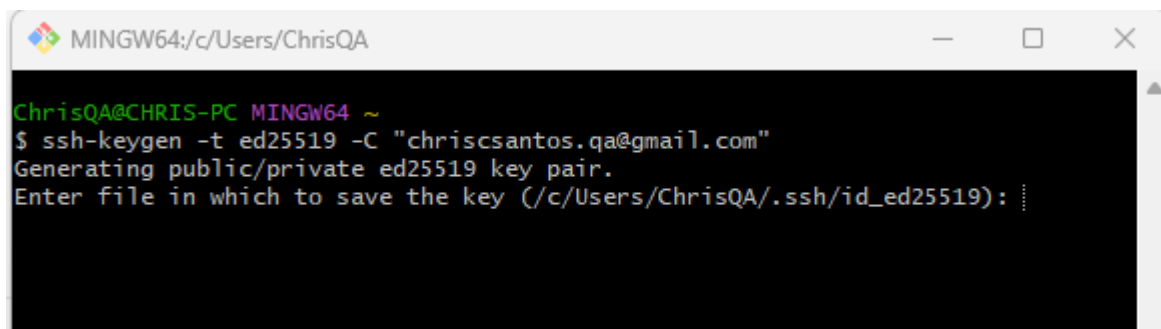
2.5 Certo, agora mantenha essa página aberta e vamos gerar uma chave **SSH** para o e-mail que você utilizou no cadastro da sua conta no **GitHub**. Primeiro você precisa já ter instalado em sua máquina o **Git**, com ele já instalado abre o menu iniciar do **Windows**, pesquise por “**Git Bash**” e clique sobre ele



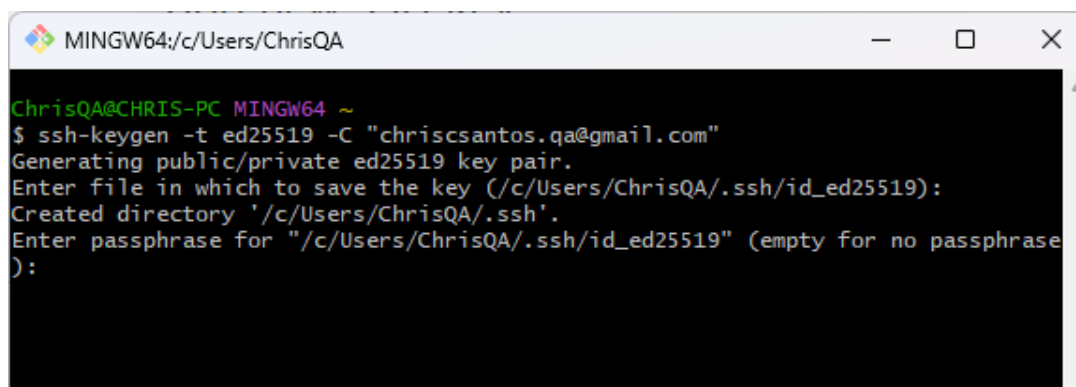
2.6 Na janela do **Git Bash** que se abrir, digite o comando abaixo e substitua o texto “**seu_email@example.com**” pelo e-mail que você utilizou no cadastro da conta do **GitHub** e aperte a tecla “**Enter**”



2.7 Agora aperte a tecla “**Enter**” novamente para confirmar o endereço onde ele irá armazenar a chave criada, não é necessário digitar nada.

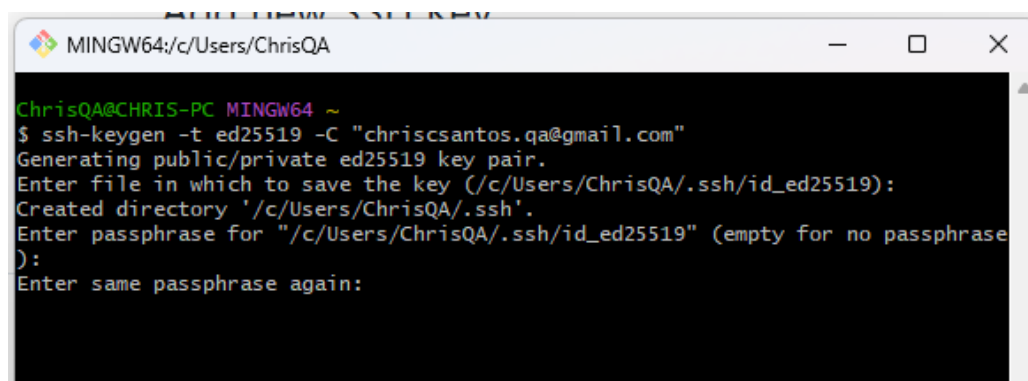


2.8 Agora aperte **“Enter”** novamente para deixar a senha em branco, não será necessário utilizar uma senha, mas caso queira digitar uma senha lembre-se de anotar ela.



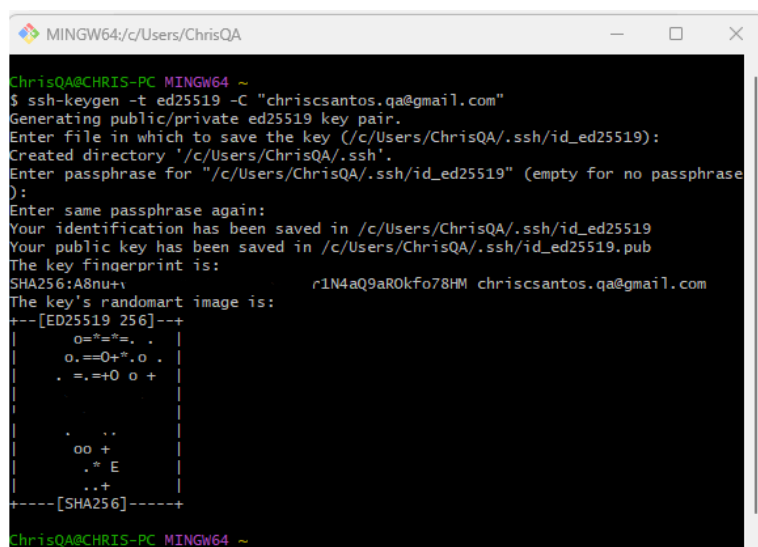
```
MINGW64:/c/Users/ChrisQA
ChrisQA@CHRIS-PC MINGW64 ~
$ ssh-keygen -t ed25519 -C "chriscsantos.qa@gmail.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/c/Users/ChrisQA/.ssh/id_ed25519):
Created directory '/c/Users/ChrisQA/.ssh'.
Enter passphrase for "/c/Users/ChrisQA/.ssh/id_ed25519" (empty for no passphrase
):
```

2.9 Aperte a tecla **“Enter”** novamente. Caso tenha digitado uma senha no passo anterior, digite a mesma senha novamente e aperte a tecla **“Enter”**



```
MINGW64:/c/Users/ChrisQA
ChrisQA@CHRIS-PC MINGW64 ~
$ ssh-keygen -t ed25519 -C "chriscsantos.qa@gmail.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/c/Users/ChrisQA/.ssh/id_ed25519):
Created directory '/c/Users/ChrisQA/.ssh'.
Enter passphrase for "/c/Users/ChrisQA/.ssh/id_ed25519" (empty for no passphrase
):
Enter same passphrase again:
```

2.10 Pronto! Após isso você irá conseguir visualizar as informações da chave criada



```
MINGW64:/c/Users/ChrisQA
ChrisQA@CHRIS-PC MINGW64 ~
$ ssh-keygen -t ed25519 -C "chriscsantos.qa@gmail.com"
Generating public/private ed25519 key pair.
Enter file in which to save the key (/c/Users/ChrisQA/.ssh/id_ed25519):
Created directory '/c/Users/ChrisQA/.ssh'.
Enter passphrase for "/c/Users/ChrisQA/.ssh/id_ed25519" (empty for no passphrase
):
Enter same passphrase again:
Your identification has been saved in /c/Users/ChrisQA/.ssh/id_ed25519
Your public key has been saved in /c/Users/ChrisQA/.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:A8nu+... r1N4aQ9aR0kfo78HM chriscsantos.qa@gmail.com
The key's randomart image is:
+--[ED25519 256]--+
|  o=*==*..  |
|  o==0+*.o  |
|  .==+0 o +  |
|              |
|  oo +      |
|  .* E      |
|  ..+       |
+-----[SHA256]-----+
ChrisQA@CHRIS-PC MINGW64 ~
```

Na imagem acima foi necessário apagar algumas informações, porém você irá receber o mesmo modelo de retorno.

```
cat ~/.ssh/id_ed25519.pub
```

MINGW64:/c:/Users/ChrisQA

```
|      oo +      |
|      . * E      |
|      ..+      |
+-----[SHA256]-----+
```

ChrisQA@CHRIS-PC MINGW64 ~

```
$ cat ~/.ssh/id_ed25519.pub
```

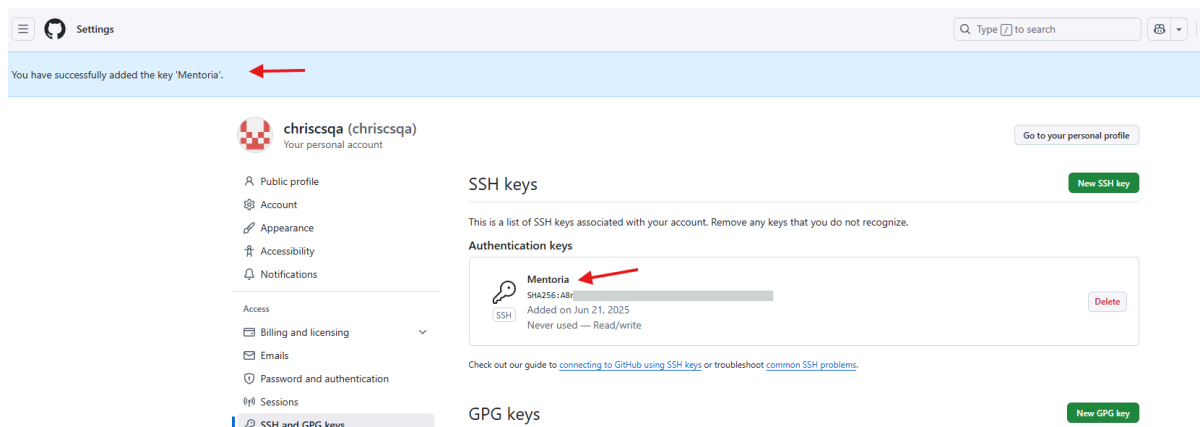


The screenshot shows a Windows terminal window with the title bar 'MINGW64; c:/Users/ChrisQA'. The terminal prompt is 'ChrisQA@CHRIS-PC MINGW64 ~'. The command '\$ cat ~/.ssh/id_ed25519.pub' has been entered, and the output is displayed, showing the contents of the public key file. A context menu is open over the terminal output, with the 'Copy' option highlighted by a red arrow. The menu also includes 'Open' and 'Paste' options, with keyboard shortcuts 'Ctrl+Ins' and 'Shift+Ins' respectively.

```
ChrisQA@CHRIS-PC MINGW64 ~  
$ cat ~/.ssh/id_ed25519.pub  
ssh-ed25519 AAAAC3NzaC1lbnRlLnQkZG90bWkzazatwKF1NY0AiAhOeeNReVG5y6g TF71K  
chriscsantos.qa@gmail.com  
  
ChrisQA@CHRIS-PC MINGW64 ~  
$ AC  
  
ChrisQA@CHRIS-PC MINGW64 ~
```

The screenshot shows the 'Add new SSH Key' page on GitHub. On the left is a sidebar with the user's profile 'chriscsq' and a list of settings: Public profile, Account, Appearance, Accessibility, Notifications, Access, Billing and licensing, Emails, Password and authentication, Sessions, SSH and GPG keys (highlighted), Organizations, Enterprises, and Moderation. The main content area is titled 'Add new SSH Key'. It contains four fields: 'Title' (containing 'Mentoria'), 'Key type' (a dropdown menu set to 'Authentication Key'), 'Key' (containing a long alphanumeric string), and a green 'Add SSH key' button at the bottom. Red arrows point to each of these four elements.

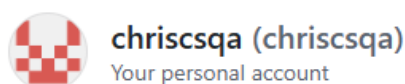
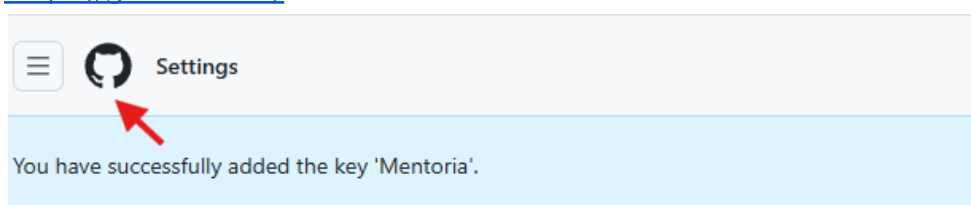
2.14 Pronto! Você receberá um alerta de sucesso e irá notar que foi inserida uma nova chave **SSH** com o “**Title**” Mentoria.



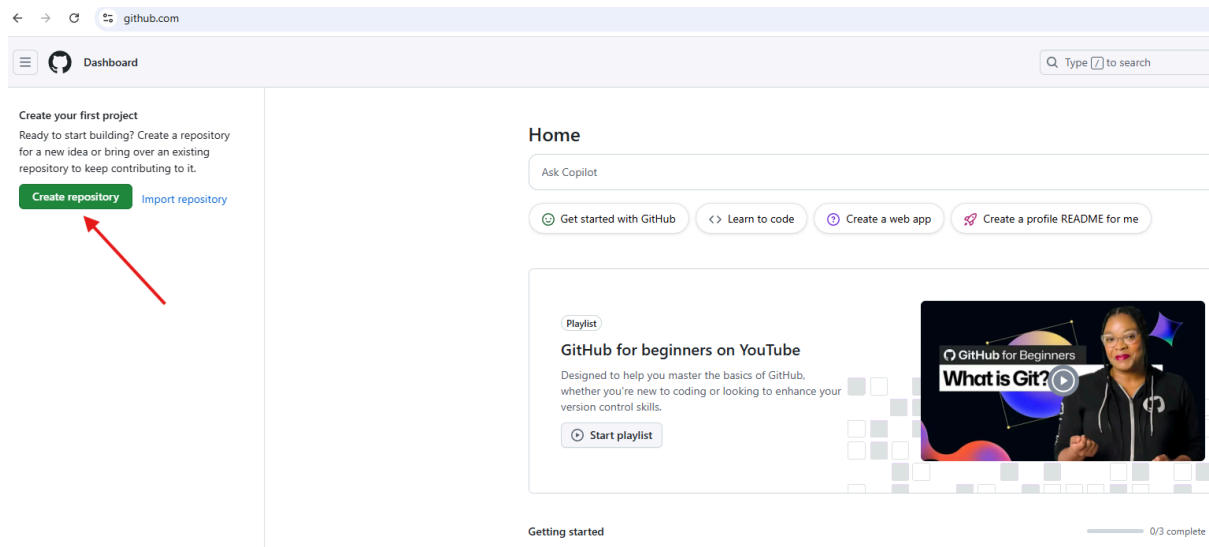
3 Criando um novo Repositório

3.1 Agora vamos criar um novo repositório para armazenar nosso projeto. Caso você não esteja na tela inicial do **GitHub**, clique no ícone dele no canto superior esquerdo ou acesse a url abaixo (**caso você não esteja logado, será necessário realizar o login novamente**)

<https://github.com/>



3.2 Agora na nossa tela inicial do **GitHub**, clique no botão “**Create repository**”



3.3 Agora vamos digitar um nome válido para o seu repositório no campo de texto “**Repository name**”

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere [Import a repository](#).

Required fields are marked with an asterisk (*).

Owner * Repository name *

 chriscsqa /

meu-repositorio is available.

Great repository names are short and memorable. Need inspiration? How about [turbo-sniffle](#) ?

3.4 Após digitar o nome do seu **Repositório**, clique no botão “**Create repository**”

Create a new repository

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk (*).

Owner * Repository name *
 chriscsq / meu-repositorio 
✔ meu-repositorio is available.

Great repository names are short and memorable. Need inspiration? How about [turbo-sniffle](#) ?

Description (optional)

- ☒  **Public**
Anyone on the internet can see this repository. You choose who can commit.
- ☐  **Private**
You choose who can see and commit to this repository.

Initialize this repository with:

- ☐ Add a README file
This is where you can write a long description for your project. [Learn more about READMEs.](#)

Add .gitignore

.gitignore template: **None**

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

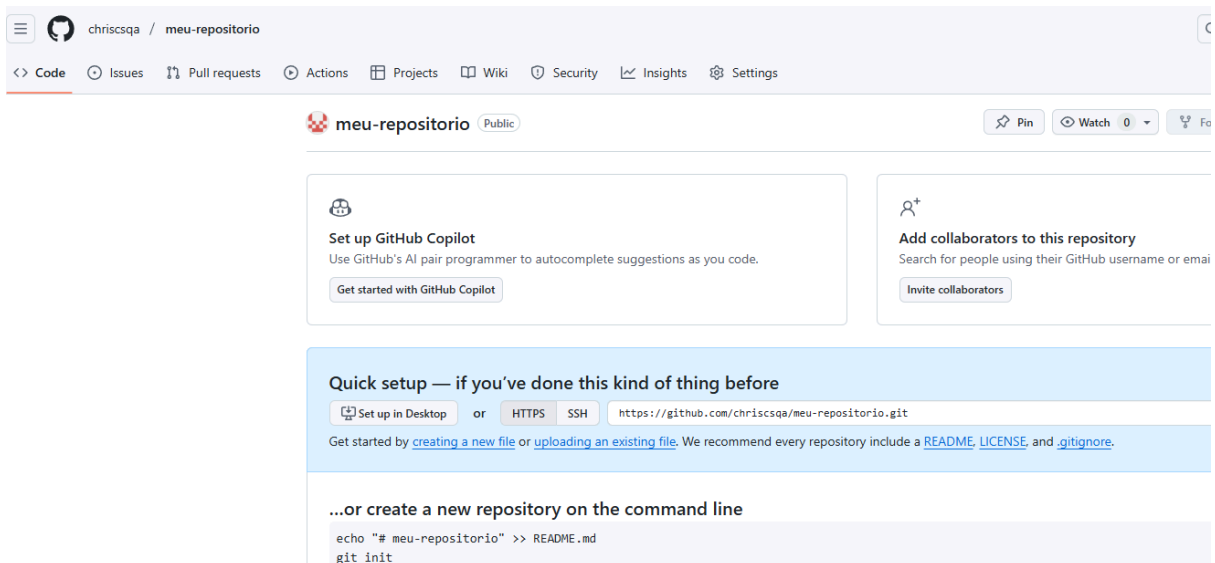
License: **None**

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

 You are creating a public repository in your personal account.

 **Create repository**

3.5 Pronto! Agora o repositório já está criado, mas agora vamos precisar **clonar(baixar)** ele para a nossa máquina

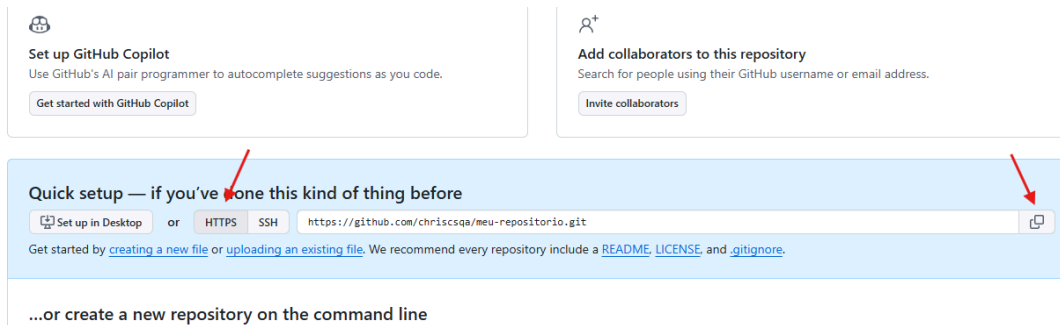


The screenshot shows the GitHub repository page for 'meu-repositorio' by user 'chriscsq'. The repository is public. The page includes a navigation bar with links to Code, Issues, Pull requests, Actions, Projects, Wiki, Security, Insights, and Settings. Below the navigation bar, there are two main sections: 'Set up GitHub Copilot' and 'Add collaborators to this repository'. The 'Set up GitHub Copilot' section includes a button 'Get started with GitHub Copilot'. The 'Add collaborators to this repository' section includes a button 'Invite collaborators'. Below these sections, there is a 'Quick setup' section with links to 'Set up in Desktop', 'HTTPS', and 'SSH', and a URL 'https://github.com/chriscsq/meu-repositorio.git'. It also includes a note about including a README, LICENSE, and .gitignore. At the bottom, there is a section titled '...or create a new repository on the command line' with a code block showing the commands to create a new repository and initialize it.

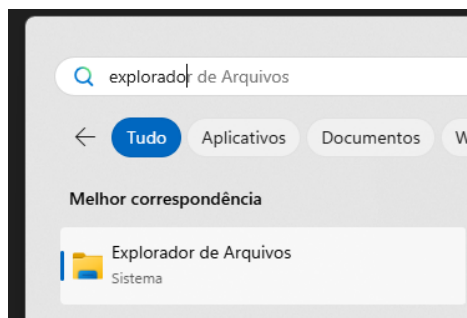
```
echo "# meu-repositorio" >> README.md
git init
```

4. Clonando o repositório criado

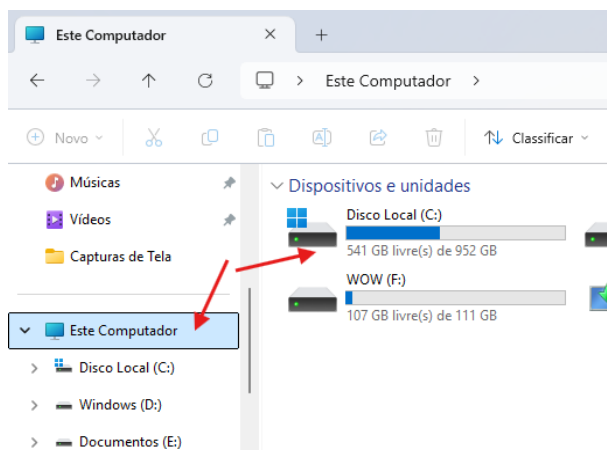
4.1 Agora, em “Quick setup” clique na opção “HTTPS” e clique no botão “Copiar” ao lado direito da caixa de texto



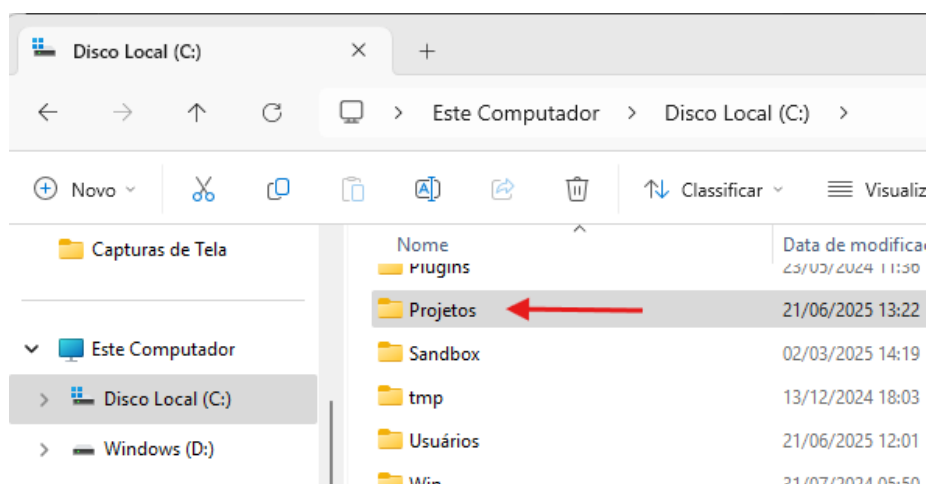
4.2 No menu iniciar do **Windows** pesquise pela ferramenta “**Explorador de Arquivos**” e clique sobre ela



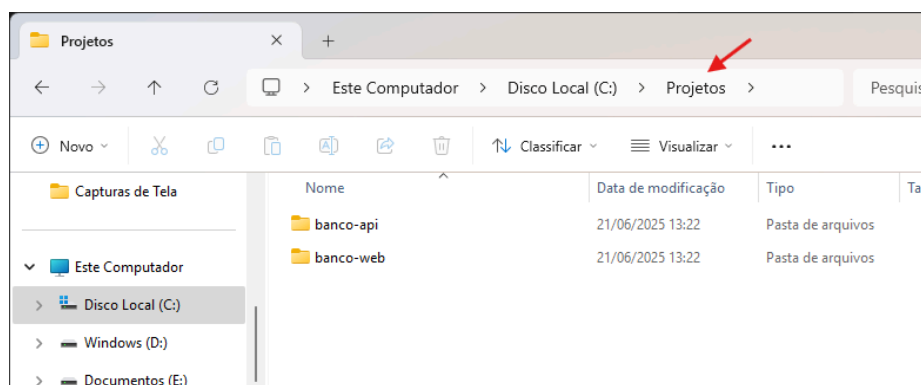
4.3 Utilizando o menu lateral esquerdo, navegue até a opção “**Este Computador**” e clique sobre ela, depois clique 2 vezes na opção “**Disco Local (C:)**”



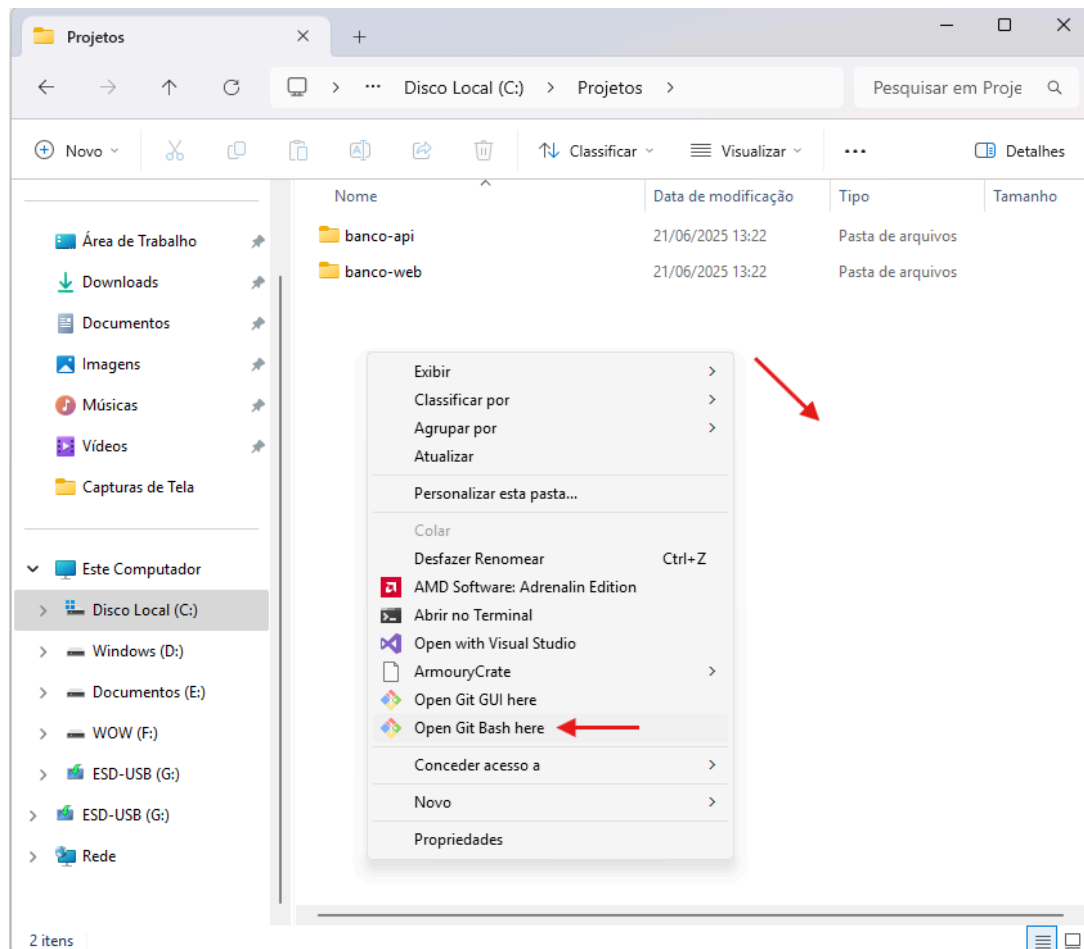
4.4 Agora dentro do “**Disco Local (C:)**” crie uma nova pasta chamada **Projetos** ou utilize a pasta já existente que foi criada no tutorial do **módulo 2**. Clique 2 vezes em cima da pasta **Projetos** para acessá-la



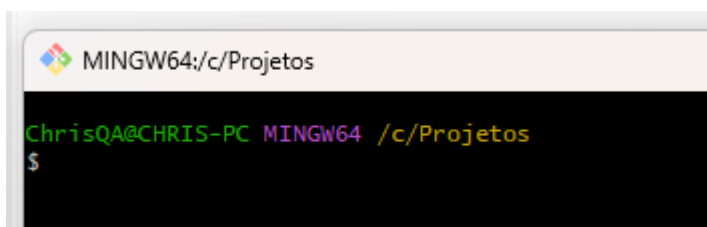
4.5 Se você criou a pasta no passo **4.4** não se preocupe pois você não terá as mesmas pastas que estão na imagem do exemplo abaixo.



4.6 Agora clique com o direito do mouse em um espaço vazio da pasta, procure pela opção “**Mostrar mais opções**” e clique em “**Git BASH here**”

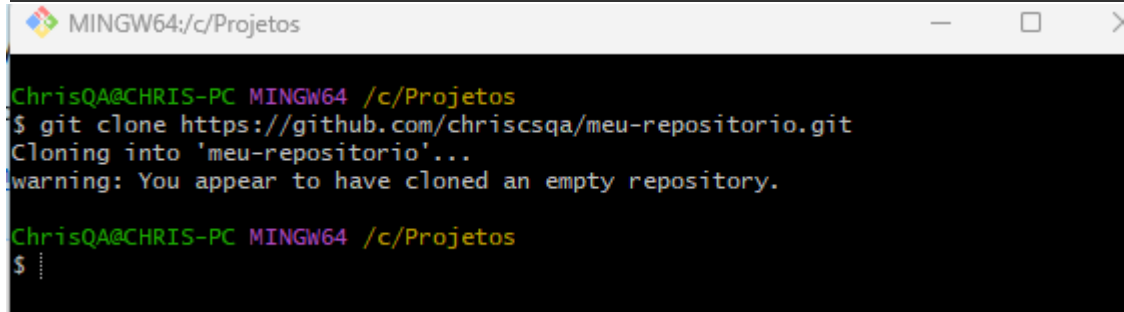


4.7 Uma janela do **Git** será aberta contendo o endereço da pasta **Projetos**



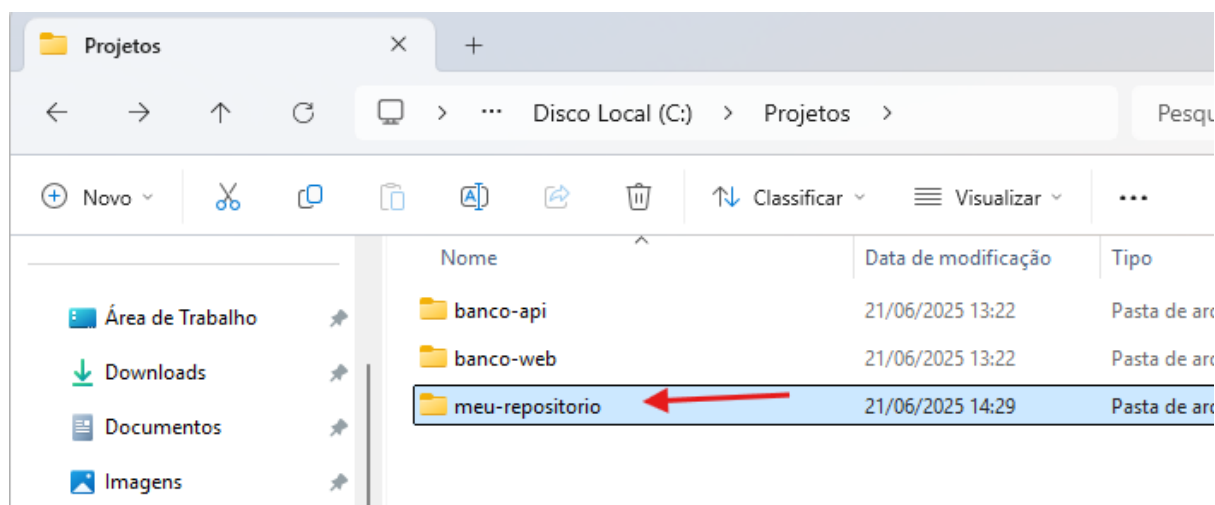
4.8 Na janela do **Git** digite o comando “**git clone**” e cole o endereço que você copiou no **GitHub** no passo **3.6**. Deverá ficar conforme imagem abaixo, com a diferença que no seu caso onde está escrito “**chriscsqa**” estará o nome do seu usuário do **GitHub**

```
git clone https://github.com/SEUUSUARIO/meu-repositorio.git
```



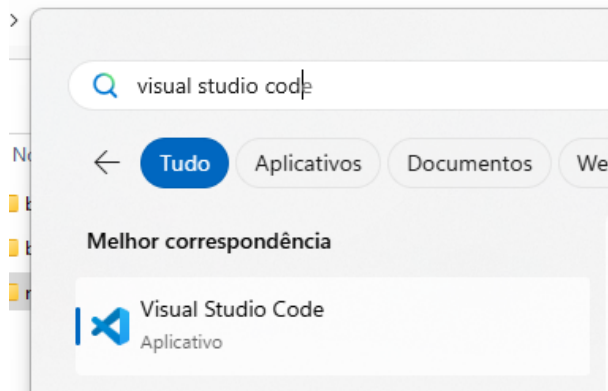
The image shows a terminal window titled 'MINGW64:/c/Projetos'. The prompt is 'ChrisQA@CHRIS-PC MINGW64 /c/Projetos'. The command entered is '\$ git clone https://github.com/chriscsqa/meu-repositorio.git'. The output is 'Cloning into 'meu-repositorio'...' followed by a warning: 'warning: You appear to have cloned an empty repository.' The prompt returns to '\$ '.

4.9 Pronto! Caso dê tudo certo você irá notar que foi criada uma pasta com o nome do seu repositório dentro da pasta **Projetos**

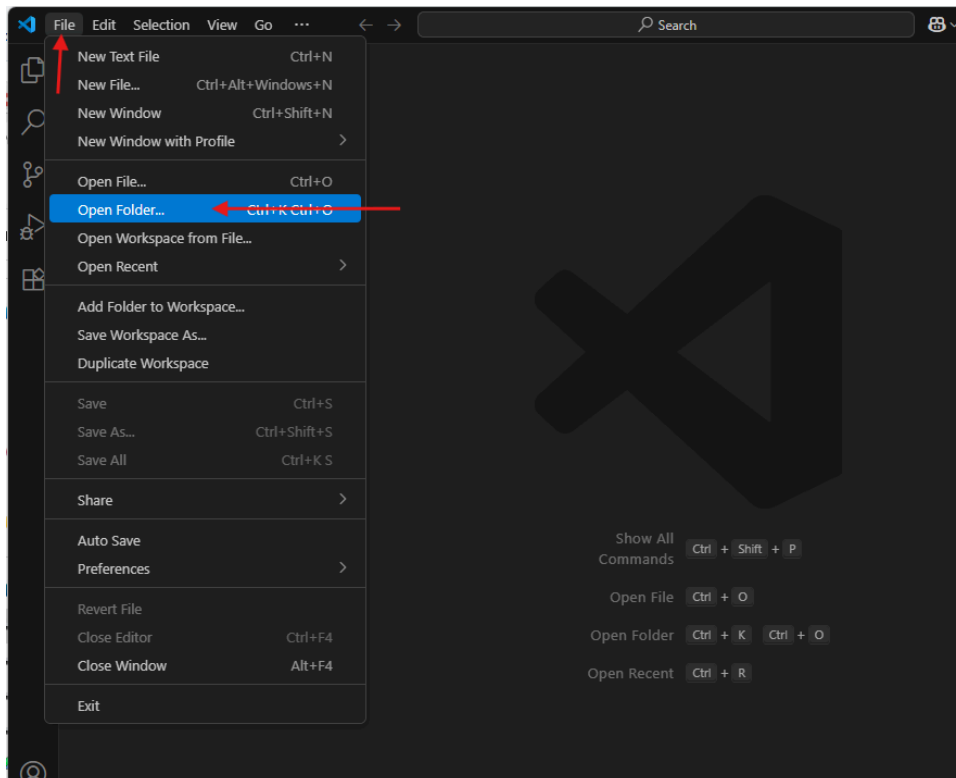


5. Interagindo com o Repositório utilizando os comandos do Git

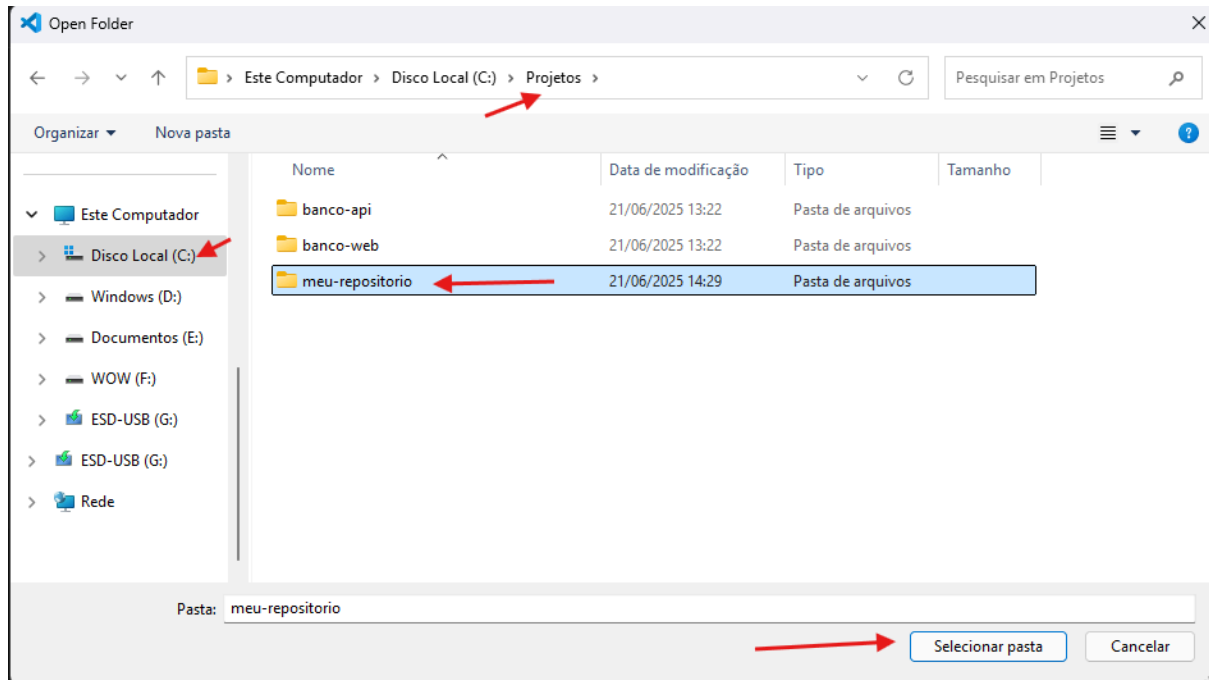
5.1 Agora vamos interagir com o **Repositório** que **clonamos(baixamos)** anteriormente utilizando a ferramenta **Visual Studio Code**. Clique no **menu iniciar** do **Windows**, pesquise por “**Visual Studio Code**” e clique sobre ele



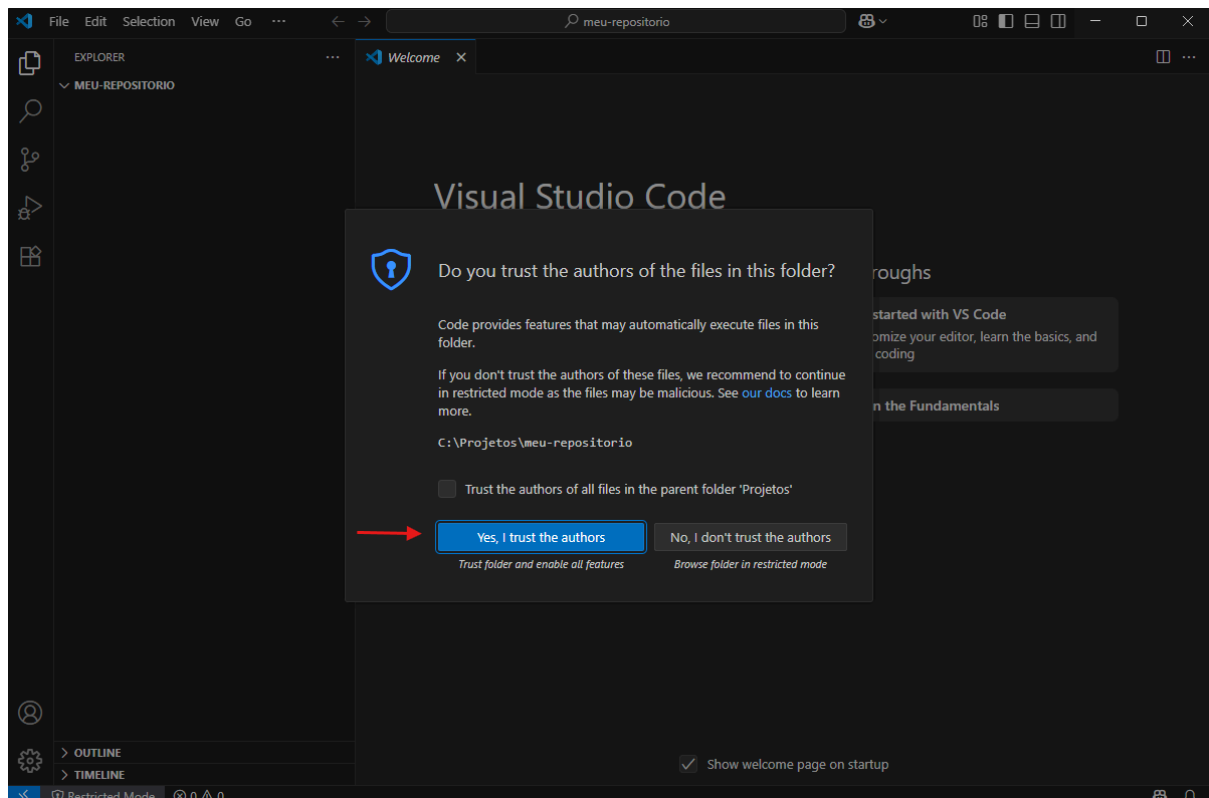
5.2 Agora na janela do Visual Studio Code clique no menu “**File**” e depois clique na opção “**Open Folder**”



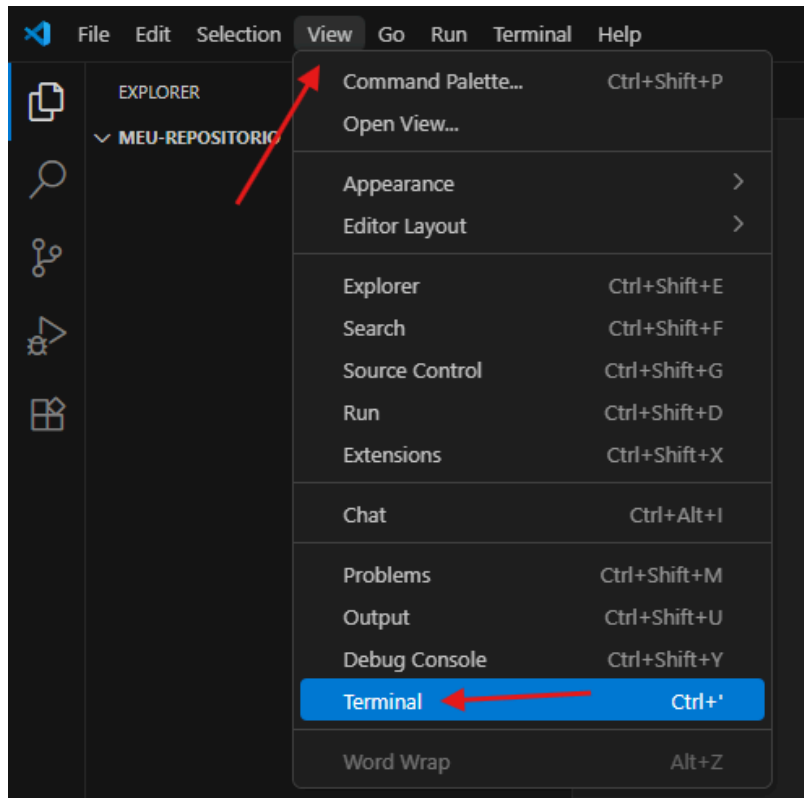
5.3 Navegue até a pasta **Projetos** onde **clonamos(baixamos)** nosso repositório, selecione a pasta do nosso novo repositório e clique no botão “**Selecionar pasta**”



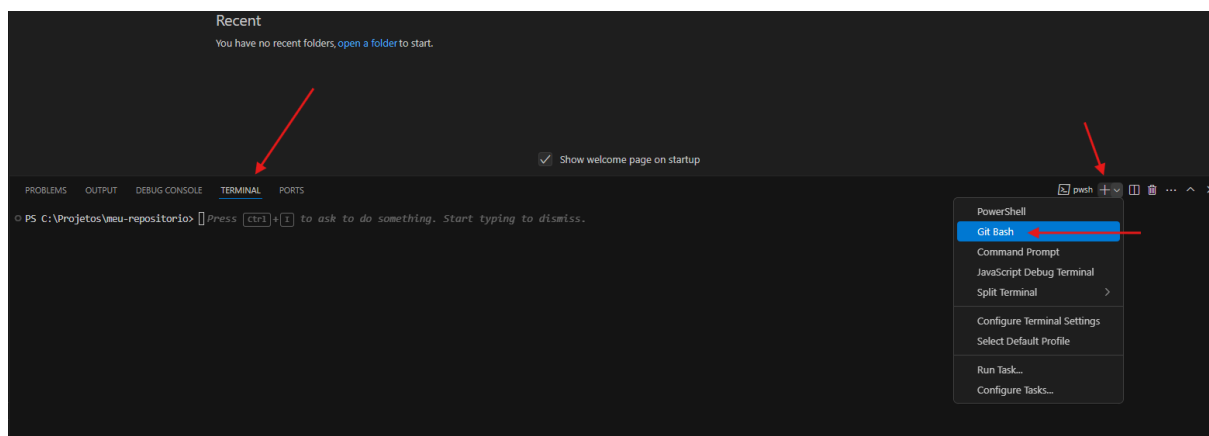
5.4 Caso o **Visual Studio Code** exiba a mensagem de alerta abaixo, apenas clique no botão “**Yes, I trust the authors**”



5.5 Agora vamos abrir o terminal para interagir com o repositório. Clique no menu “**View**” e clique na opção “**Terminal**”



5.6 Na divisão que se abrir na parte inferior da nossa janela do Visual Studio Code clique no botão “+” e clique na opção “**Git Bash**”



5.7 Agora vamos criar uma **Branch(ramificação)** para criarmos o nosso arquivo **JavaScript**. Digite o comando “**git checkout -b**” e o nome que você quer dar a sua branch e aperte a tecla “**Enter**” para executar o comando, siga o exemplo abaixo

```
git checkout -b primeira-branch
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (main)
● $ git checkout -b primeira-branch
  Switched to a new branch 'primeira-branch'

ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (primeira-branch)
○ $
```

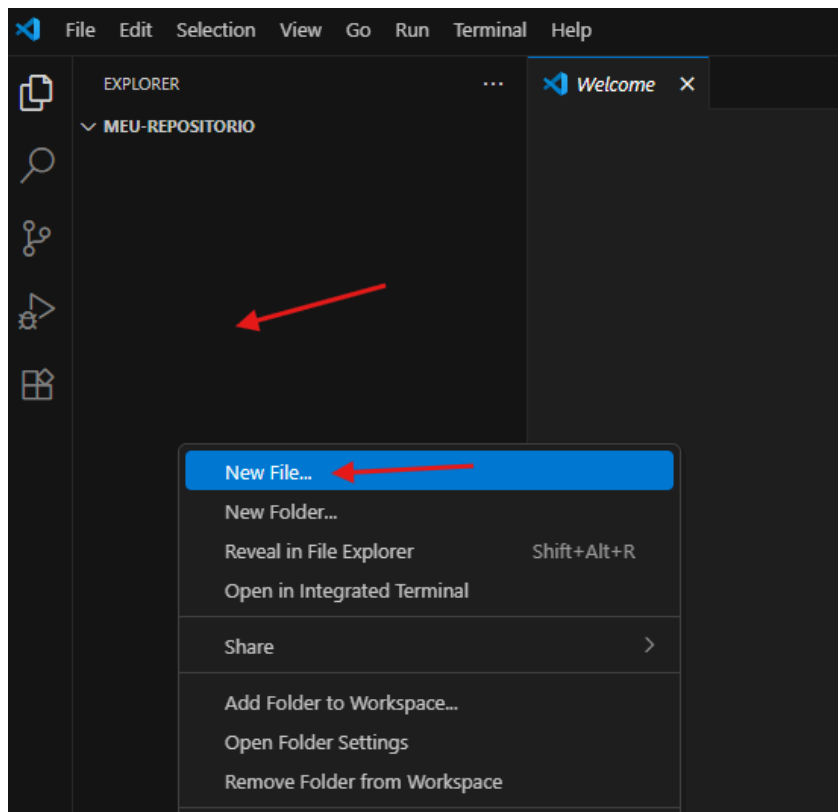
5.8 Você irá notar uma diferença ao lado do endereço da pasta do projeto que estamos trabalhando, na imagem utilizada de exemplo neste tutorial o trecho do texto que está em azul indica o nome da branch que estamos naquele exato momento, e com isso é possível ver que antes de criar a nova branch nós estávamos na branch “**main**” e após executar o comando nós estamos na branch “**primeira-branch**” que foi criada através do comando “**git checkout -b nomeDaBranch**”

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

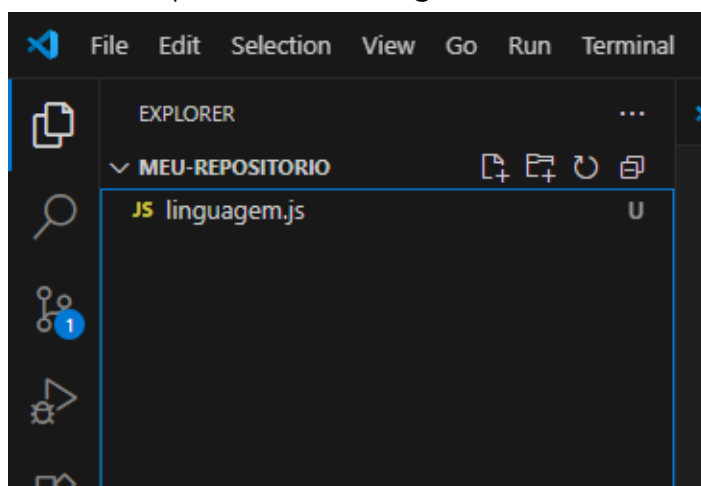
```
ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (main)
● $ git checkout -b primeira-branch
  Switched to a new branch 'primeira-branch'

ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (primeira-branch)
○ $
```

5.9 Agora clique em um espaço vazio no menu do nosso projeto, clique na opção **“New File”** e crie um arquivo chamado **“linguagem.js”**

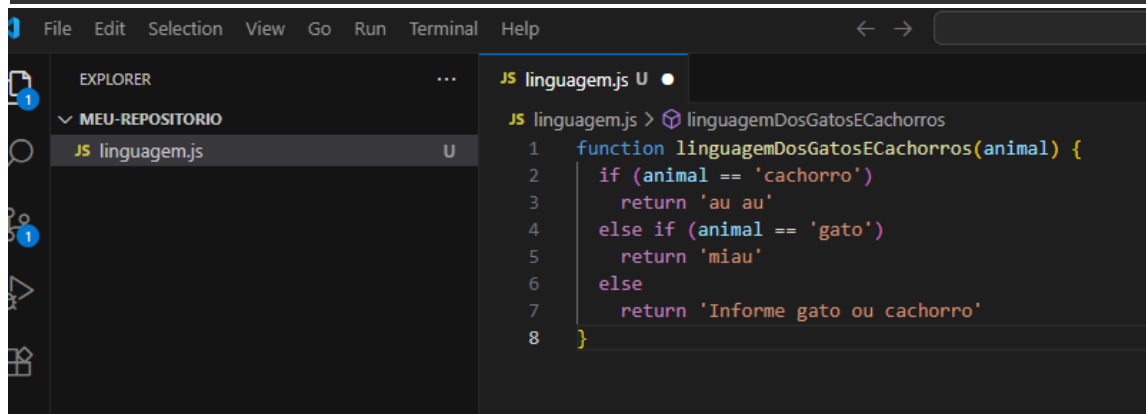


5.10 Seu arquivo ficará da seguinte forma

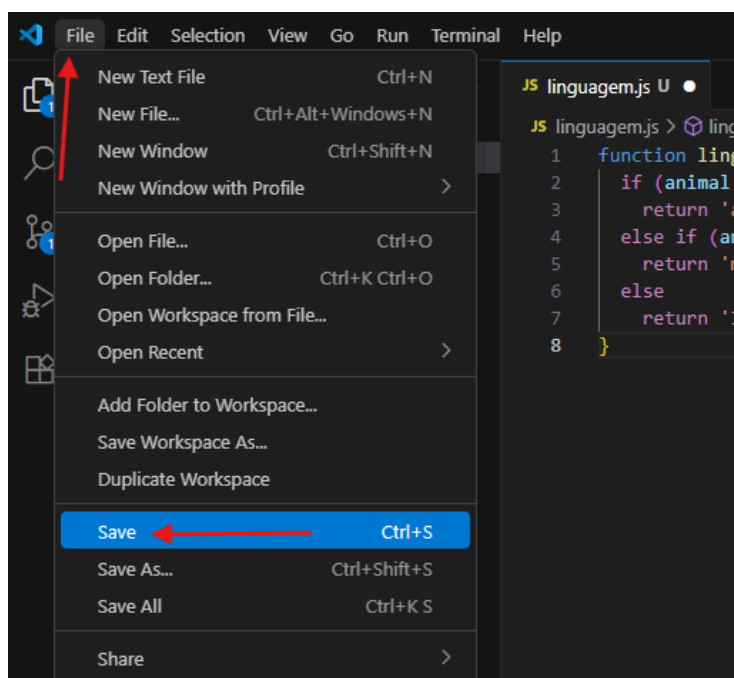


5.11 Agora clique no arquivo “**linguagem.js**” para que ele seja aberto e cole o trecho do código abaixo dentro do arquivo

```
function linguagemDosGatosECachorros(animal) {  
  if (animal == 'cachorro')  
    return 'au au'  
  else if (animal == 'gato')  
    return 'miau'  
  else  
    return 'Informe gato ou cachorro'  
}
```



5.12 Agora salve seu arquivo clicando no menu “**File**” e depois clicando na opção “**Save**”



5.13 Agora que temos o primeiro arquivo do nosso novo repositório, vamos enviá-lo para o **GitHub** para que ele seja armazenado na plataforma. No terminal digite o comando abaixo para preparar o nosso novo arquivo criado para que ele seja enviado ao repositório no **GitHub**, após digitar aperte a tecla **"Enter"**

```
git add linguagem.js
```

```
ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (primeira-branch)
● $ git add linguagem.js

ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (primeira-branch)
○ $
```

5.14 Agora precisamos inserir um comentário utilizando o comando **"git commit -m"** que descreva o arquivo que estamos enviar, digite o comando abaixo no terminal

```
git commit -m "Adicionando a função linguagem"
```

Caso seja sua primeira vez utilizando o **Git**, você irá receber um erro conforme exemplo abaixo

```
ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (primeira-branch)
⊗ $ git commit -m "Adicionando a função linguagem"
Author identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'ChrisQA@CHRIS-PC.(none)')

ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (primeira-branch)
○ $
```

Para resolver isso, siga os próximos passos. Caso você já tenha realizado esta configuração em outro momento, apenas avance para o passo **5.15**

5.14.1 Copie o comando abaixo e substitua o trecho “**you@example.com**” pelo e-mail que você utilizou no cadastro do **GitHub**

```
git config --global user.email "SEU@EMAIL.COM"
```

```
ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (primeira-branch)
● $ git config --global user.email "chriscsantos.qa@gmail.com"

ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (primeira-branch)
○ $
```

5.14.2 Agora copie o próximo comando e substitua o “**Your Name**” pelo seu nome

```
git config --global user.name "SEU NOME"
```

```
ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (primeira-branch)
● $ git config --global user.name "Chris"

ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (primeira-branch)
○ $
```

5.14.3 Pronto! Após isso execute novamente o comando abaixo para criamos um comentário para o arquivo que estamos enviando para o nosso repositório no **GitHub**

```
git commit -m "Adicionando a função linguagem"
```

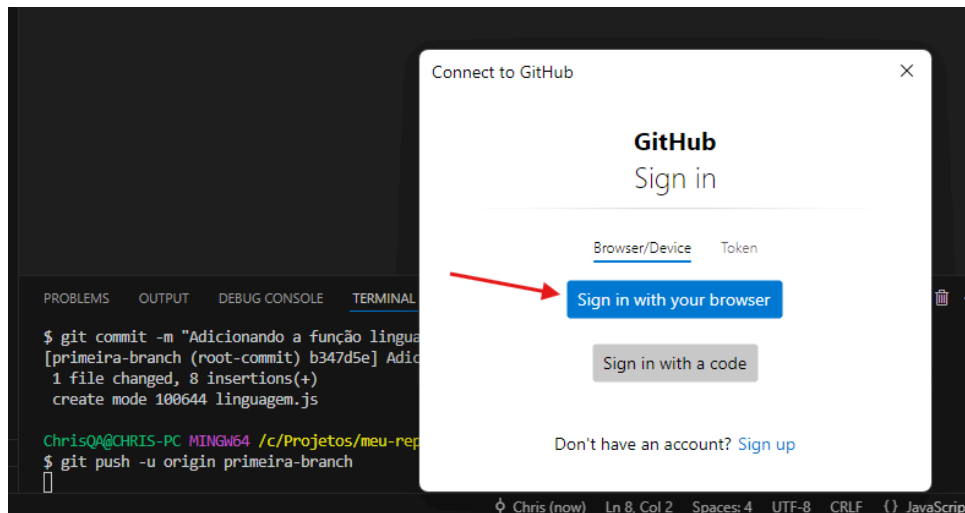
```
ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (primeira-branch)
● $ git commit -m "Adicionando a função linguagem"
[primeira-branch (root-commit) b347d5e] Adicionando a função linguagem
1 file changed, 8 insertions(+)
create mode 100644 linguagem.js

ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (primeira-branch)
○ $
```

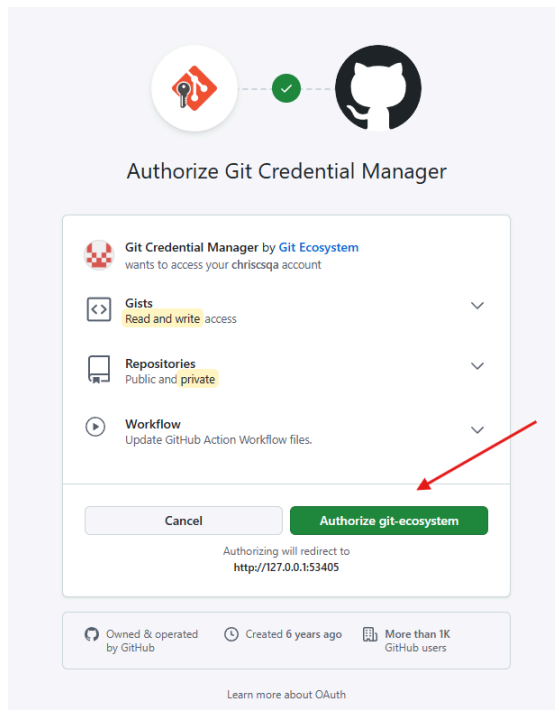
5.15 Agora vamos enviar o nosso novo arquivo para o repositório no **GitHub**. Digite o comando “**git push -u origin**” e o nome da branch para que nas próximas vezes que você for enviar novos arquivos ou modificações de arquivos para essa branch você utilize apenas o comando “**git push**” para o envio

```
git push -u origin primeira-branch
```

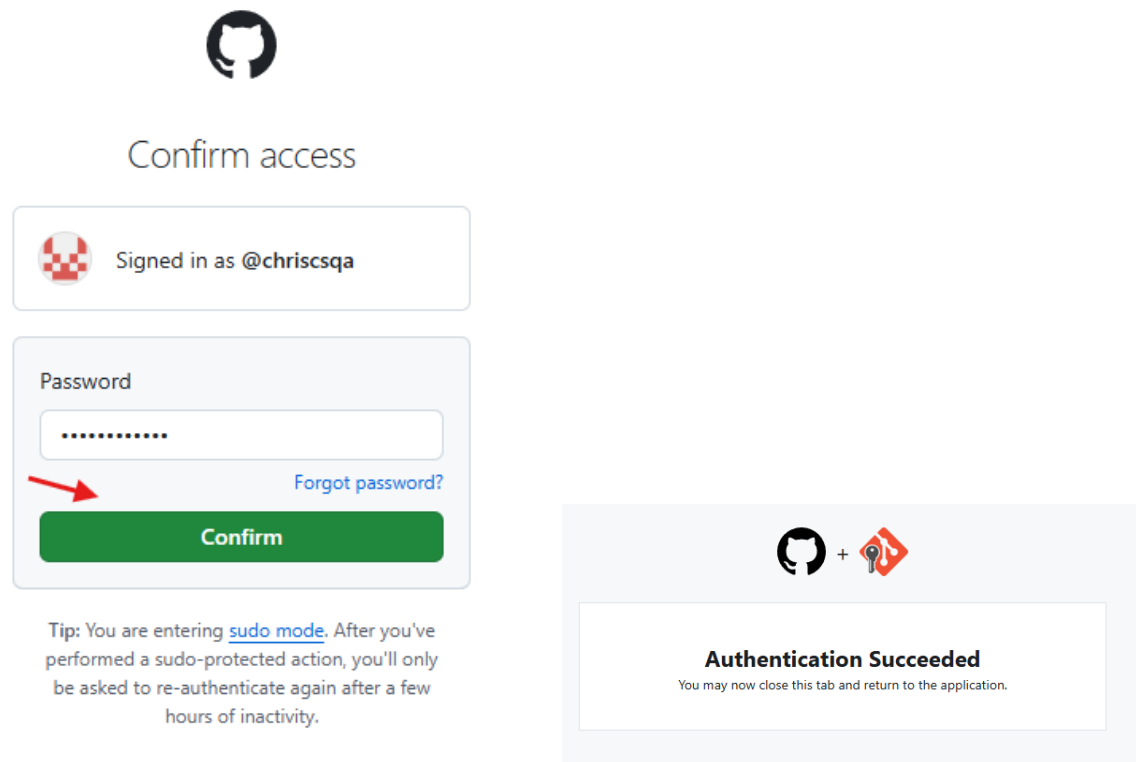
Caso seja sua primeira vez, irá surgir uma janela pedindo pra você se conectar ao **GitHub**, basta apenas clicar no botão “**Sign in with your browser**”



Na janela que se abrir clique no botão “**Authorize git-ecosystem**”



E após isso digite a senha da sua conta **GitHub** que você utilizou em seu cadastro e clique no botão **“Confirm”**



Confirm access

Signed in as @chriscsqa

Password

.....

[Forgot password?](#)

Confirm

Tip: You are entering [sudo mode](#). After you've performed a sudo-protected action, you'll only be asked to re-authenticate again after a few hours of inactivity.

Authentication Succeeded
You may now close this tab and return to the application.

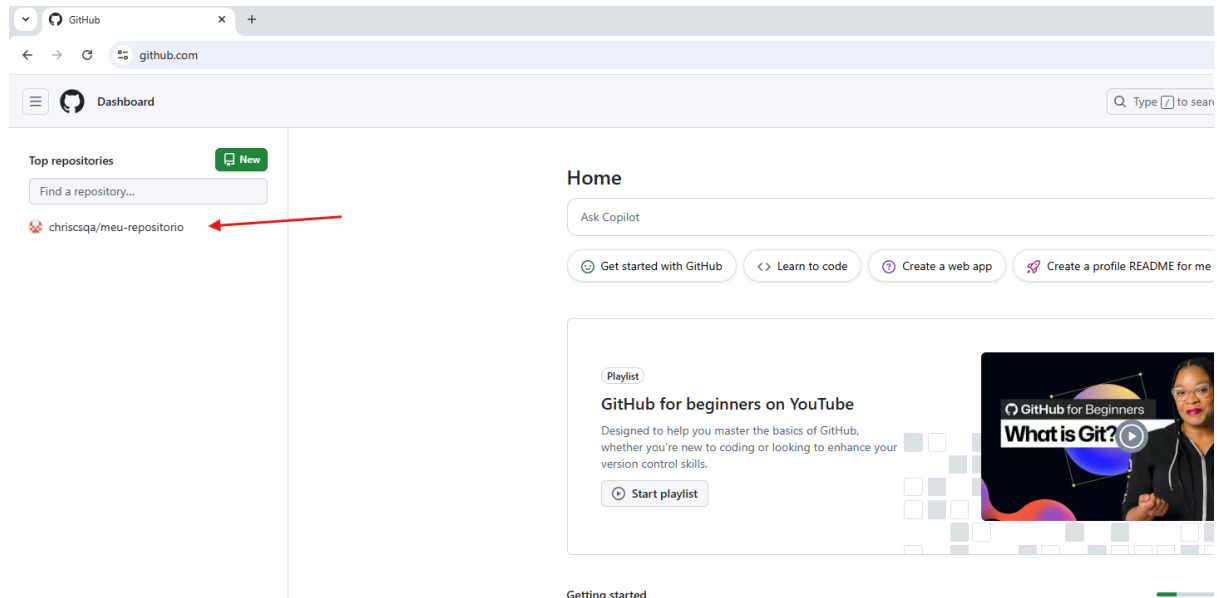
5.16 Pronto! Você irá ver no terminal todo processo de envio com sucesso ao seu repositório no **GitHub**

```
ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (primeira-branch)
$ git push -u origin primeira-branch
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 346 bytes | 346.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/chriscsqa/meu-repositorio.git
 * [new branch]      primeira-branch -> primeira-branch
branch 'primeira-branch' set up to track 'origin/primeira-branch'.

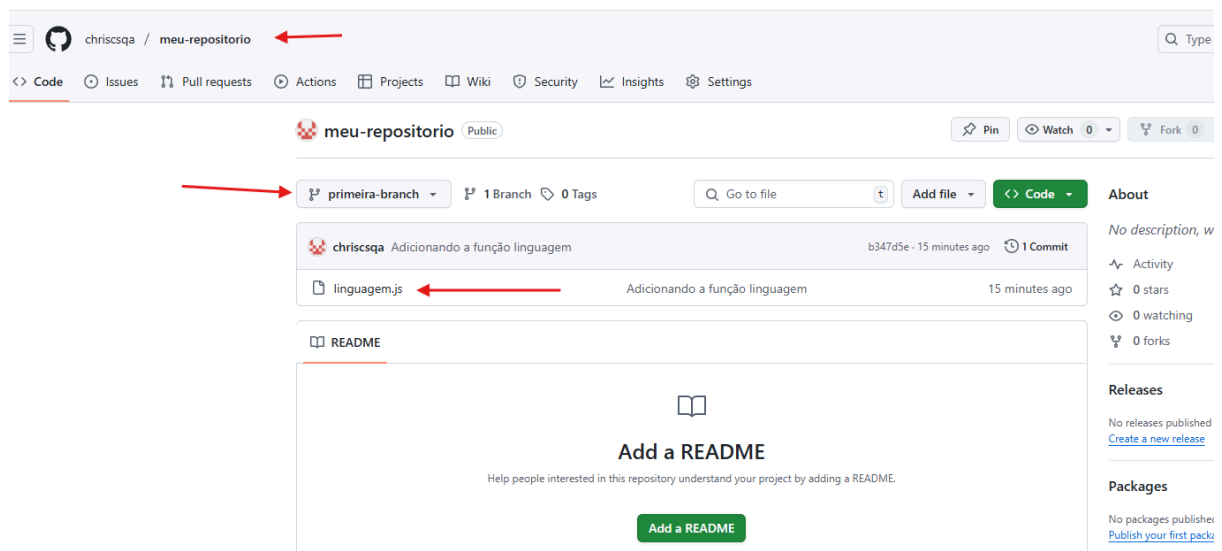
ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (primeira-branch)
$
```

5.17 Para confirmar que o envio foi um sucesso, abra o site do **GitHub** abaixo (faça login se necessário) e clique no repositório que criamos que agora está visível na página inicial do **GitHub**

<https://github.com/>



5.18 Estando tudo certo você irá conseguir ver o arquivo “**linguagem.js**” que criamos localmente no **Visual Studio Code** disponível dentro do seu repositório online no **GitHub**



5.19 Agora vamos criar uma nova branch para poder melhorar ou modificar o nosso arquivo “**linguagem.js**” sem que isso afete o arquivo original

5.20 Voltando ao nosso **Visual Studio Code**, na divisão inferior na aba **Terminal**, digite o comando **“git checkout -b”** e o nome da nova branch para criá-la

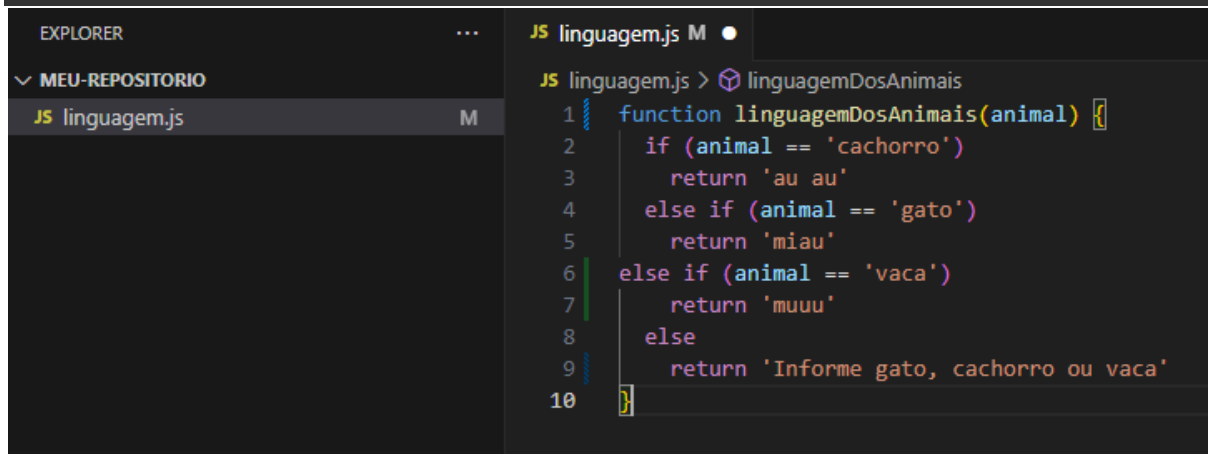
```
git checkout -b segunda-branch
```

```
ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (primeira-branch)
$ git checkout -b segunda-branch
Switched to a new branch 'segunda-branch'

ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (segunda-branch)
$ git pull
```

Agora vamos alterar a function do arquivo **“linguagem.js”**, copie o código abaixo e substitua pelo código já existente

```
function linguagemDosAnimais(animal) {
  if (animal == 'cachorro')
    return 'au au'
  else if (animal == 'gato')
    return 'miau'
  else if (animal == 'vaca')
    return 'muuu'
  else
    return 'Informe gato, cachorro ou vaca'
}
```



5.21 **Salve** seu arquivo clicando no menu **“File”** e depois clicando na opção **“Save”**



5.22 Agora no nosso terminal digite o comando “**git add**” e o nome do arquivo para iniciarmos a preparação de envio ao repositório no **GitHub**

```
git add linguagem.js
```

```
ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (segunda-branch)
● $ git add linguagem.js
```

5.23 Agora vamos inserir um comentário para a modificação que fizemos utilizando o comando “**git commit -m**” e a descrição da modificação conforme exemplo abaixo

```
git commit -m "Função renomeada e adicionado a opção vaca no retorno"
```

```
ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (segunda-branch)
● $ git commit -m "Função renomeada e adicionado a opção vaca no retorno"
[segunda-branch 319f61b] Função renomeada e adicionado a opção vaca no retorno
1 file changed, 3 insertions(+), 1 deletion(-)
```

Lembre-se sempre de colocar o texto da descrição entre aspas duplas

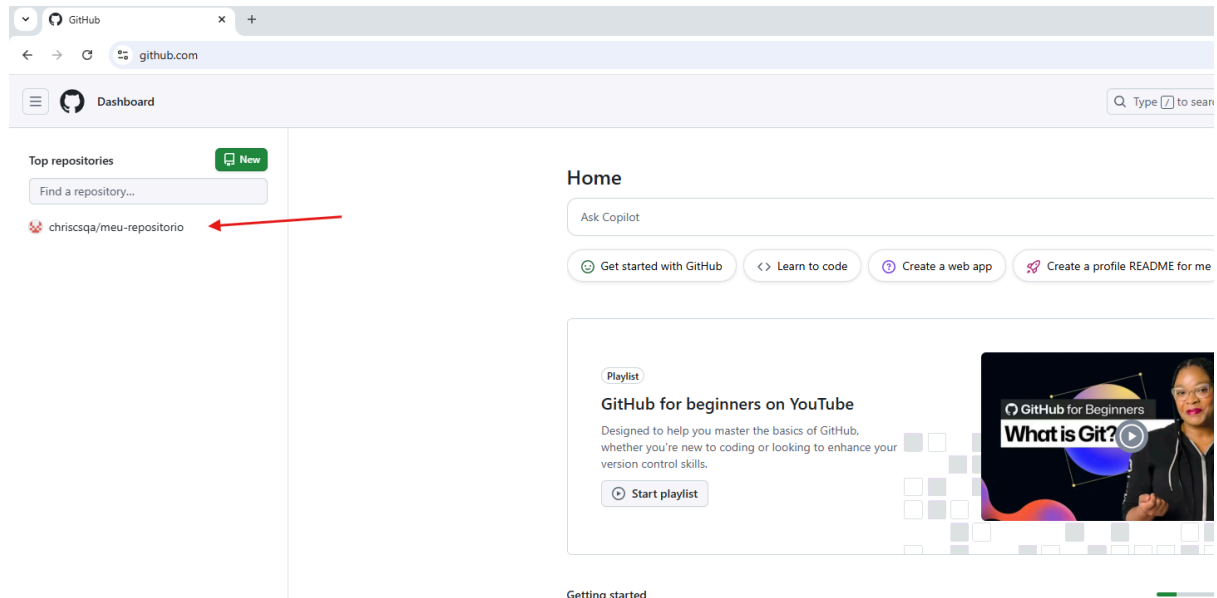
5.24 Agora vamos enviar a nossa nova branch e a modificação que fizemos na function para o repositório no **GitHub** utilizando o comando “**git push -u origin**” e o nome da branch, conforme exemplo abaixo

```
git push -u origin segunda-branch
```

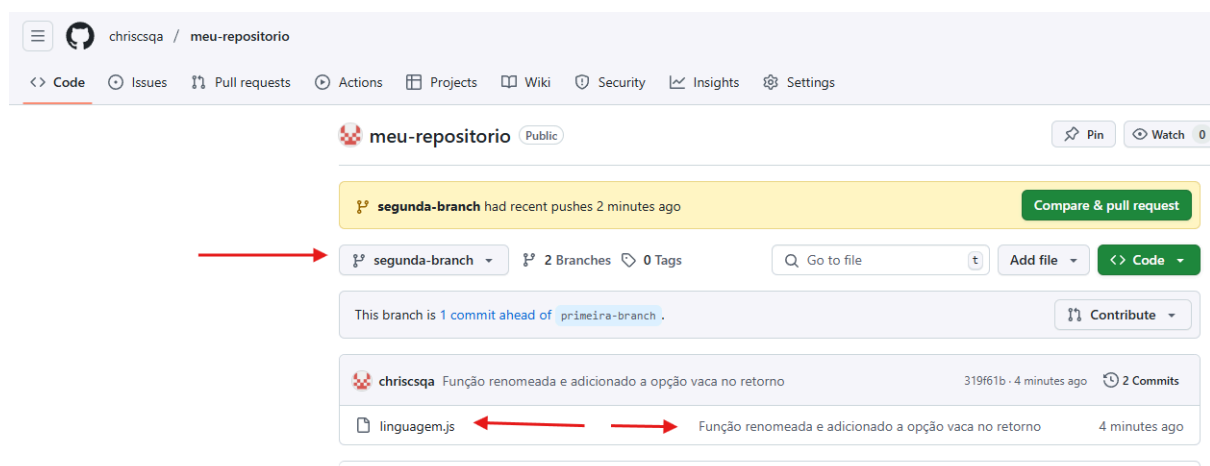
```
ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (segunda-branch)
● $ git push -u origin segunda-branch
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (3/3), 356 bytes | 356.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
remote:
remote: Create a pull request for 'segunda-branch' on GitHub by visiting:
remote:   https://github.com/chriscsqa/meu-repositorio/pull/new/segunda-branch
remote:
To https://github.com/chriscsqa/meu-repositorio.git
 * [new branch]      segunda-branch -> segunda-branch
branch 'segunda-branch' set up to track 'origin/segunda-branch'.
●
ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (segunda-branch)
○ $
```

5.25 Para confirmar que o envio foi um sucesso, abra o site do **GitHub** abaixo(**faça login se necessário**) e clique no repositório que criamos que agora está visível na página inicial do **GitHub**

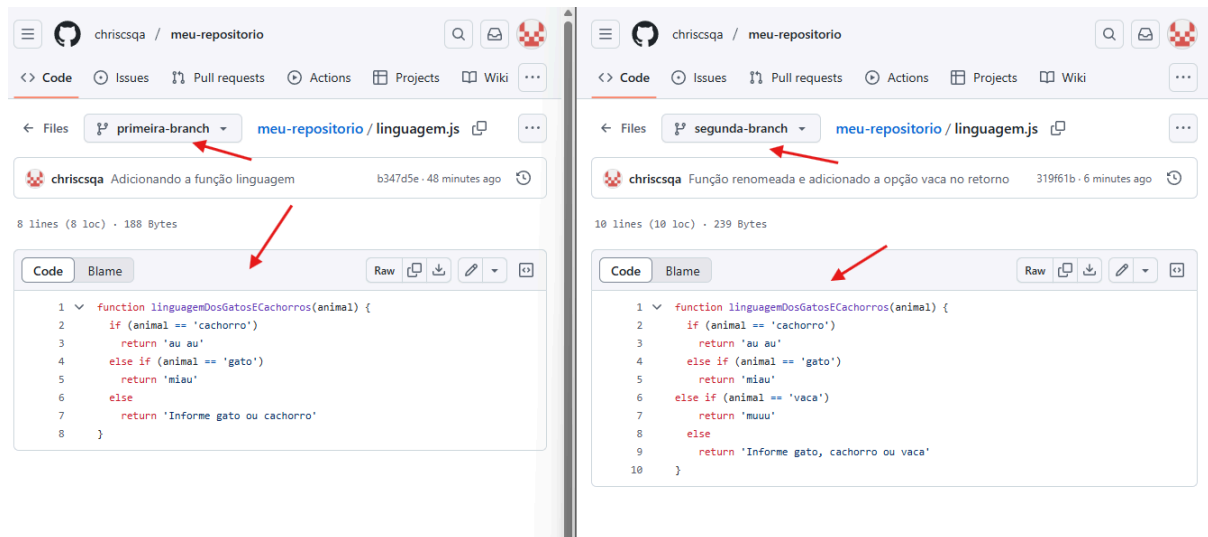
<https://github.com/>



5.26 Estando tudo certo você irá conseguir ver o arquivo **“linguagem.js”** que modificamos localmente no **Visual Studio Code** disponível dentro da nova branch em seu repositório online no **GitHub**



5.27 Agora vamos comparar os arquivos das duas branches que criamos. Você irá notar que mesmo modificando o arquivo “**linguagem.js**” na **segunda-branch**, o arquivo original criado dentro da **primeira-branch** permanece intacto após as alterações, permitindo que seja possível evoluir o código sem afetar o código já existente que era funcional.



6. Navegando entre as Branchs

6.1 Agora que temos **2 branches criadas**, vamos navegar entre elas pelo **Terminal** do **Visual Studio Code** para comparar o arquivo “**linguagem.js**” entre ambas

6.2 Dentro do **Terminal** do **Git Bash**, digite o comando “**git checkout**” e o nome da branch existente para alternar entre elas, após digitar aperte a tecla “**Enter**”

```
git checkout primeira-branch
```

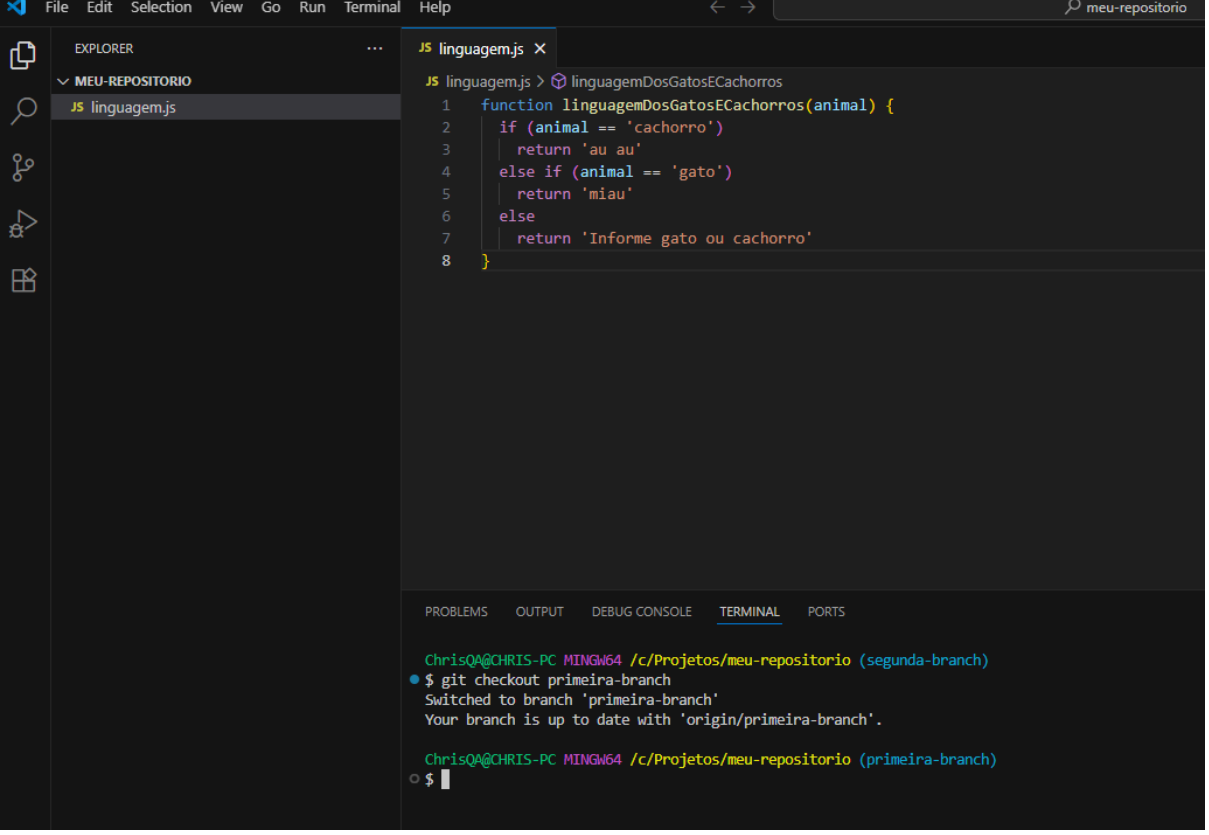
```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (segunda-branch)
● $ git checkout primeira-branch
Switched to branch 'primeira-branch'
Your branch is up to date with 'origin/primeira-branch'.

ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (primeira-branch)
○ $
```

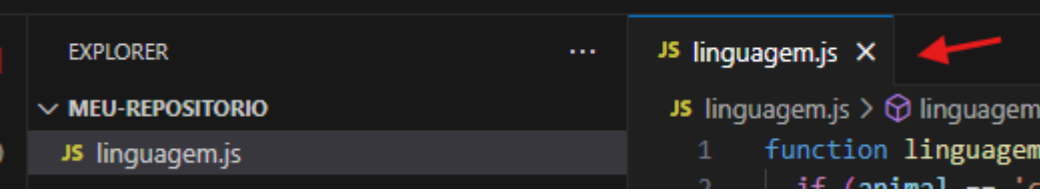
Você irá notar que ele irá sair da **segunda-branch** e vai para a **primeira-branch**

6.3 Agora clique em cima do arquivo “**linguagem.js**” para abri-lo e note que seu conteúdo é o mesmo que criamos pela primeira vez.



```
File Edit Selection View Go Run Terminal Help
EXPLORER
MEU-REPOSITORIO
  JS linguagem.js
JS linguagem.js
1 function linguagemDosGatosECachorros(animal) {
2   if (animal == 'cachorro')
3     return 'au au'
4   else if (animal == 'gato')
5     return 'miau'
6   else
7     return 'Informe gato ou cachorro'
8 }
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (segunda-branch)
• $ git checkout primeira-branch
Switched to branch 'primeira-branch'
Your branch is up to date with 'origin/primeira-branch'.
ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (primeira-branch)
o $
```

6.4 Agora feche a aba do arquivo “**linguagem.js**”, digite o comando “**git checkout**” e o nome da outra branch, e aperte a tecla “**Enter**”

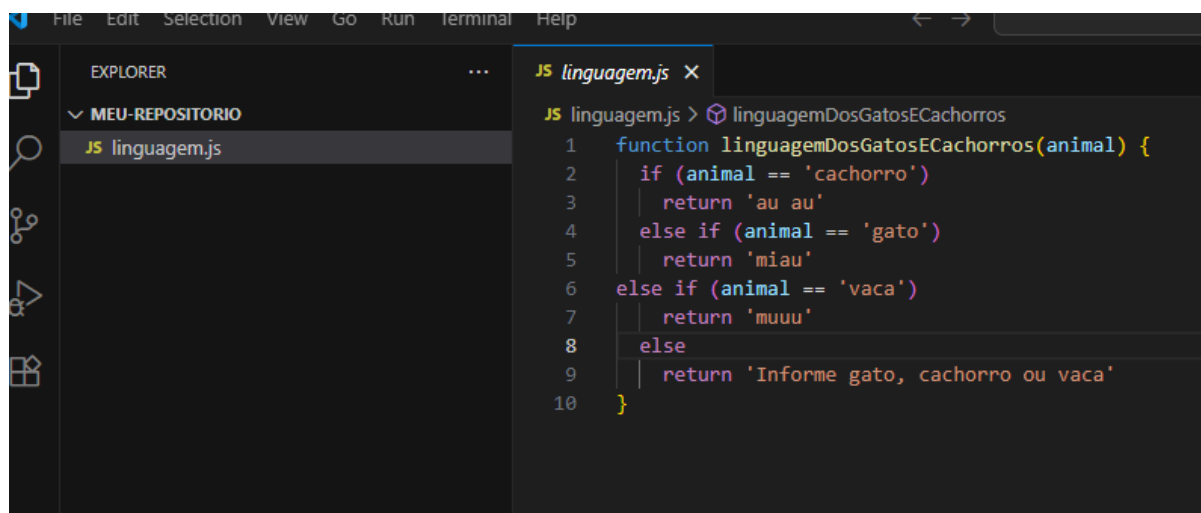


```
EXPLORER
MEU-REPOSITORIO
  JS linguagem.js
JS linguagem.js
1 function linguagem
2   if (animal == 'c
```

```
git checkout segunda-branch
```

```
ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (primeira-branch)
• $ git checkout segunda-branch
Switched to branch 'segunda-branch'
Your branch is up to date with 'origin/segunda-branch'.
```

6.5 Pronto! Agora ao selecionar o arquivo “**linguagem.js**” você irá conseguir visualizar o arquivo modificado novamente



6.6 Lembre-se caso o arquivo não seja carregado normalmente ao trocar de branch, utilize o comando “**git pull**” para que você atualize os arquivos localmente da branch, com os arquivos existentes no repositório online do **GitHub**

```
ChrisQA@CHRIS-PC MINGW64 /c/Projetos/meu-repositorio (segunda-branch)
• $ git pull
Already up to date.
```

Referência

Caso você queira conhecer mais sobre os comandos existentes do Git, e quais ações eles realizam, acesse o site abaixo e navegue pelos comandos para conhecer um pouco mais sobre cada um.

<https://git-scm.com/docs>